Hurricane Erin: Building WRF and WPS on Amarel

L. Fernando Pareja-Roman

Numerical Modeling of the Ocean and Atmosphere I
16:712:603:01

Fall 2026

Email: pareja at marine.rutgers.edu

This exercise prepares the software needed to generate atmospheric inputs and run a Hurricane Erin simulation. We use Intel's `ifx` Fortran compiler and the software environment provided by Amarel's `roms` module.

How to use this guide

Work through the sections in order. Each stage prepares something needed by the next stage.

Blocks labeled **Run in the terminal** contain commands to copy into your Amarel terminal. Blocks labeled **Edit inside the file** contain settings to enter in the named script. Expected output is shown separately so you can compare it with your screen.

Replace `YOUR_NETID` with your own account name wherever it appears.

---

1. Understand the workflow

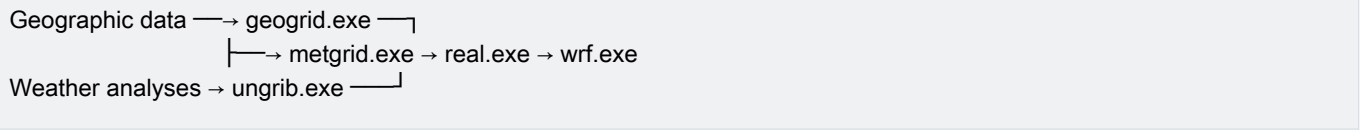
Three source repositories are involved:

Software	Role in this exercise
ROMS	Supplies the build scripts used to organize the WRF and WPS builds. Its ocean model will be needed for later coupled experiments.
WRF	Supplies `real.exe`, which creates atmospheric initial and boundary conditions, and `wrf.exe`, which runs the atmospheric model.
WPS	Prepares geographic and meteorological fields for `real.exe`.

The build order is **download the sources → build WRF → build WPS**. WPS needs libraries produced by the WRF build. We need the ROMS source and its scripts now; we do not need to compile ROMS before WRF.

Later, input preparation and forecasting follow this sequence:

Expected display or file layout:



- `geogrid.exe` defines the domain and prepares terrain and other geographic fields.

- `ungrib.exe` decodes the atmospheric GRIB files.
- `metgrid.exe` interpolates the atmospheric fields onto the model grid.
- `real.exe` creates WRF initial and boundary conditions.
- `wrf.exe` runs the forecast.

Building software creates these executables. It does not download the weather data or run the simulation. Using ROMS' build scripts also does not, by itself, create the final ROMS–WRF coupled application.

---

2. Create your working folders

Before downloading anything, we need a place for the source code and a separate place for our experiment. This makes it easier to tell software files from the files produced by a simulation.

The guide uses `nmoa_2026` as the main folder. You may choose another name, but you must use it consistently in all commands and script paths.

`$HOME` means **your own home directory**, such as `/home/your_netid`. The commands below use it so each student works in their own account.

Run in the terminal:

```
mkdir -p "$HOME/nmoa_2026/Erin/prep"
mkdir -p "$HOME/nmoa_2026/Erin/run"
cd "$HOME/nmoa_2026"
```

After downloading the source code, the layout will be:

Expected display or file layout:

```
nmoa_2026/
├── ROMS/            ROMS source and build scripts
├── WRF/             WRF source
├── WPS/             WPS source
└── Erin/            Our experiment
    ├── prep/        Prepare atmospheric input files
    └── run/         WRF build and forecast work
```

The source folders are kept together so they are easy to locate. Experiment settings and products belong under `Erin`. Another experiment can have its own folder alongside `Erin`.

We build WRF from `Erin/run` because ROMS' script uses the directory from which it is launched as the application directory. Its build products go into `Erin/run/Build_wrf`.

Although `real.exe` is built with WRF, we will run it during input preparation in `Erin/prep`.

---

3. Load the software environment

Now that the experiment folders exist, we prepare the compiler and libraries that will be used throughout both builds. Use the same environment for WRF and WPS so their libraries are compatible.

On Amarel, run:

Run in the terminal:

```
module purge
module load roms
```

The `roms` module supplies Intel compilers, MPI, NetCDF, and related libraries. You do not need to load oneAPI or MPI individually for this exercise.

Check the environment:

Run in the terminal:

```
module list
command -v ifx icx mpiifx mpiicx
nf-config --version
nc-config --version
```

The tested installation used NetCDF-Fortran 4.6.3 and NetCDF-C 4.10.0 with Intel oneAPI 2025.3 and Intel MPI. The exact installation paths may change as Amarel is updated.

If `module load roms` reports that the module is unavailable, the course module directory needs to be made visible first:

Run in the terminal:

```
module use /projects/dmcs_1/sw/modulefiles/Core
module load roms
```

---

4. Download ROMS, WRF, and WPS

With the environment loaded, we can download the source repositories. Downloading gives us the code; compilation will turn that code into executable programs.

Run these commands once from the main working folder. If a destination folder already exists, inspect it before repeating a clone command.

Run in the terminal:

```
cd "$HOME/nmoa_2026"
```

Download ROMS

We use the ROMS source revision that supplied the tested build scripts:

Run in the terminal:

```
git clone --branch develop --recurse-submodules \
    https://github.com/myroms/roms.git ROMS

cd ROMS
git checkout 31cabd318c9379429c365c308b9e0f2ee166a6fc
git submodule update --init --recursive
cd ..
```

Download WRF

Use the **myroms fork**, its `Coupling` branch, and the revision corresponding to the WRF 4.5.2 source used for this exercise:

Run in the terminal:

```
git clone --branch Coupling --recurse-submodules \
    https://github.com/myroms/WRF.git WRF

cd WRF
git checkout ee6e88c9e0507e8d88e35187a618f617a885ca39
git submodule update --init --recursive
head -n 1 README
cd ..
```

The README should identify **WRF Model Version 4.5.2**.

Checking out a specific commit may produce a “detached HEAD” message. This is expected: we are deliberately using a fixed source revision so everyone starts from the same code.

Set the MYNN COARE option before compiling

Our eventual experiment will couple WRF, which represents the atmosphere, with ROMS, which represents the ocean. The two models exchange information at the sea surface. The treatment of momentum, heat, and moisture exchange at that interface is therefore an important part of the experiment.

Later in the course, we will configure ROMS to use the **COARE 3.5 bulk-flux formulation**. To prepare WRF for that same experiment, we also select **COARE 3.5 in its MYNN surface-layer implementation now**. This keeps the intended COARE version consistent between the atmospheric and ocean configurations.

We make this change before compiling because `COARE_OPT` is a constant in the Fortran source code. Its value becomes part of the executable during compilation. Changing the text file afterward will not change an executable that has already been built; we would need to rebuild WRF.

Open the source file:

Run in the terminal:

```
nano "$HOME/nmoa_2026/WRF/phys/module_sf_mynn.F"
```

Search for `COARE_OPT`. Set its declaration to:

Edit inside the file:

```
  REAL, PARAMETER :: COARE_OPT=3.5  ! 3.0 or 3.5
```

This source setting is one part of the coupled-model configuration. Later, we will select the MYNN surface-layer physics in WRF's namelist, configure COARE 3.5 in ROMS, and specify how the coupled system computes and exchanges surface fluxes. Selecting the same COARE version does not by itself guarantee identical fluxes: the inputs and coupling choices also matter.

Save before compiling. A later change to this source setting requires rebuilding WRF.

Download WPS

Run in the terminal:

```
cd "$HOME/nmoa_2026"

git clone --branch v4.7.0 --recurse-submodules \
    https://github.com/wrf-model/WPS.git WPS
```

These are the source versions used in the classroom build. Do not substitute a different branch or release during this exercise without checking that the build instructions still apply.

---

5. Copy the ROMS build scripts

Now that all three source folders are available, we copy the build scripts into the experiment. These copies let us enter the paths and compiler choices for this particular setup.

Copy the scripts into the experiment directories:

Run in the terminal:

```
cd "$HOME/nmoa_2026"

cp ROMS/ESM/build_wrf.sh Erin/run/
cp ROMS/ESM/build_wps.sh Erin/prep/
```

Edit these copies. They hold the paths and compiler settings for this experiment.

In the following examples, `/home/YOUR_NETID` is a placeholder. Replace it with your actual home directory. To see that directory, run:

Run in the terminal:

```
printf '%s\n' "$HOME"
```

The lines beginning with `export` below belong **inside the build scripts**. Edit the existing lines in the files; do not enter these settings separately in the terminal.

---

6. Configure the WRF build script

We will build WRF first because WPS needs libraries created during the WRF build.

The file `build_wrf.sh` is a shell script: a sequence of commands that configures WRF, compiles its Fortran and C code, and organizes the resulting libraries and executable programs. We copied it from ROMS because ROMS provides this build workflow.

The script comes with example directory paths from another setup. Those paths do not automatically change when we copy the script. We must replace them with the locations of the two repositories we downloaded:

- `ROMS_SRC_DIR` must point to the entire `ROMS` folder. The script uses supporting files and helper scripts from that repository, including the scripts that organize the completed WRF build.
- `WRF_ROOT_DIR` must point to the entire `WRF` folder. This contains the WRF source code and its `configure` and `compile` programs, which the build script will run.

These two settings point to the downloaded repositories at the top of `nmoa_2026`. Neither should point to `Erin/run`.

We will launch the script from `Erin/run`. It records that working directory and places the compiled WRF files in `Erin/run/Build_wrf`, with the executable programs in `Build_wrf/Bin`.

Open the copy of the script that belongs to this experiment:

Run in the terminal:

```
cd "$HOME/nmoa_2026/Erin/run"
nano build_wrf.sh
```

Enter your paths and compiler settings

Inside the editor, find the existing lines that assign `ROMS_SRC_DIR` and `WRF_ROOT_DIR`. Replace their example paths with your own paths. An **absolute path** starts at `/` and gives the full location of a folder.

For example, if your home directory is `/home/abc123`, the first path would be `/home/abc123/nmoa_2026/ROMS`. In the lines below, replace `YOUR_NETID` with your account name:

Edit inside the file:

```
export ROMS_SRC_DIR=/home/YOUR_NETID/nmoa_2026/ROMS
export WRF_ROOT_DIR=/home/YOUR_NETID/nmoa_2026/WRF
```

Next, find the compiler and MPI settings in the same file. `FORT` selects the Fortran compiler; use Intel's `ifx`. `which_MPI` selects the MPI implementation used to build the parallel model; use `oneapi` to match the environment supplied by `module load roms`.

Edit inside the file:

```
export FORT=ifx
export which_MPI=oneapi
```

There are **two active assignments** of `which_MPI` in the script: one near the beginning and another in the compiler settings. Change both to `oneapi`. Update or remove the old OpenMPI comment so it matches the new setting.

Keep `USE_MPI`, `USE_MPIF90`, `USE_NETCDF`, and `USE_NETCDF4` enabled, and keep `USE_MY_LIBS=no` so the loaded environment supplies the library paths.

Add the Intel oneAPI MPI wrappers

The supplied script has a block beginning:

Find this line in the file:

```
if [ "${which_MPI}" == "intel" ]; then
```

Near the end of configuration, replace that entire small block, through its matching `fi`, with:

Edit inside the file:

```
  if [ "${which_MPI}" == "intel" ]; then
    perl -i -pe 's/^DM_FC(\s*)=(\s*)mpif90/DM_FC$1=$2mpiifort/' ${WRF_ROOT_DIR}/configure.wrf
    perl -i -pe 's/^DM_CC(\s*)=(\s*)mpicc/DM_CC$1=$2mpiicc/' ${WRF_ROOT_DIR}/configure.wrf
  elif [ "${which_MPI}" == "oneapi" ]; then
    perl -i -pe 's/^DM_FC\s*=.*/DM_FC = mpiifx/' ${WRF_ROOT_DIR}/configure.wrf
    perl -i -pe 's/^DM_CC\s*=.*/DM_CC = mpiicx/' ${WRF_ROOT_DIR}/configure.wrf
  fi
```

Preserve the surrounding script, including the next `fi` that closes the larger configuration section.

This addition tells the generated WRF configuration to use Intel's MPI compiler wrappers. The wrappers invoke the compilers with the MPI libraries needed for parallel execution.

Save and check the shell syntax:

Run in the terminal:

```
bash -n build_wrf.sh
```

No output means the shell syntax check passed; it does not yet prove that compilation will succeed.

---

7. Optional: keep the build running with tmux

`tmux` lets a terminal session remain active when you disconnect. Start it **before** launching a long build:

Run in the terminal:

```
tmux new -s erin-wrf
```

Inside the session, load the environment and return to the application directory:

Run in the terminal:

```
module load roms
cd "$HOME/nmoa_2026/Erin/run"
```

To detach later, press **Ctrl+b**, release both keys, then press **d**.

Reconnect with:

Run in the terminal:

```
tmux attach -t erin-wrf
```

Reconnect to the **same Amarel host** where you started tmux. Record its name with `hostname`; sessions on another login node will not appear there.

Tmux keeps a terminal alive; it does not allocate computing resources. Follow the course Amarel allocation instructions and use at least ten allocated CPUs for the `-j 10` command below.

---

8. Build WRF

The WRF build script is now configured. We can launch it from the experiment's run directory and answer the compiler and nesting questions.

From `Erin/run`, start the build:

Run in the terminal:

```
./build_wrf.sh -j 10
```

`-j 10` allows up to ten compilation jobs at once. It does not set the processor count for the later forecast.

The script first configures WRF, so compilation does not start immediately. Watch for the following two prompts and respond as described below.

Choose the compiler

You will see a list of compiler and parallelization options. Enter **78** for the tested WRF version:

Expected display or file layout:

```
76. (serial)  77. (smpar)  78. (dmpar)  79. (dm+sm)   INTEL (ifx/icx) : oneAPI LLVM
```

`dmpar` means distributed-memory parallelism using MPI. If the numbering differs, select by the **Intel ifx/icx, oneAPI LLVM, dmpar** label before continuing.

Choose nesting support

Enter **1**:

Expected display or file layout:

```
Compile for nesting? (1=basic, 2=preset moves, 3=vortex following) [default 1]: 1
```

This enables basic nesting support. It does not require the later simulation to use multiple domains.

What to expect

Allow **about an hour** for compilation. Hardware availability, filesystem load, and compiler settings affect the time. Leave the build running until it returns to the terminal prompt.

Expect many lines of compiler output. Warnings can occur during a successful build. Errors such as missing include files, missing libraries, or undefined references require investigation.

The build script can continue to its final housekeeping steps after compilation errors, so reaching the end of the output is not sufficient evidence of success.

---

9. Check the WRF build

When the build returns to the terminal prompt, check its products before moving on. WPS depends on the WRF library, so a failed WRF build must be resolved first.

Check the four expected executables:

Run in the terminal:

```
cd "$HOME/nmoa_2026/Erin/run"
ls -lh Build_wrf/Bin/{wrf,real,ndown,tc}.exe
```

Check the NetCDF I/O library needed by WPS:

Run in the terminal:

```
test -s Build_wrf/external/io_netcdf/libwrfio_nf.a \
    && echo "WRF NetCDF I/O library exists" \
    || echo "STOP: WRF NetCDF I/O library is missing"
```

Near the end of the terminal output, look for:

Expected display:

```
Executables successfully built
```

Confirm this message together with the executables and library above. If any are missing, stop here and review the terminal output for errors before starting WPS.

Also confirm the generated compiler and NetCDF settings:

Run in the terminal:

```
grep -E '^(DM_FC|DM_CC|NETCDFPATH|NETCDF4_DEP_LIB)[[:space:]]*=' \
    "$HOME/nmoa_2026/WRF/configure.wrf"
```

Expect `mpiifx`, `mpiicx`, and the NetCDF installations provided by the current `roms` module. Do not proceed to WPS if the required WRF executables or I/O library are missing.

---

10. Configure the WPS build script

Now that WRF has compiled successfully, we move on to WPS. Its build script needs both the WPS source directory and the WRF build directory we just created.

Run in the terminal:

```
cd "$HOME/nmoa_2026/Erin/prep"
nano build_wps.sh
```

Enter your paths and compiler settings

Edit the existing settings:

Edit inside the file:

```
export WPS_ROOT_DIR=/home/YOUR_NETID/nmoa_2026/WPS
export WPS_SRC_DIR=${WPS_ROOT_DIR}
export WRF_DIR=/home/YOUR_NETID/nmoa_2026/Erin/run/Build_wrf

export FORT=ifx
export which_MPI=oneapi
```

Again, change **both active assignments** of `which_MPI`. Keep `USE_MY_LIBS=no`.

`WRF_DIR` must point to the WRF **build directory** because that is where the ROMS build script arranged the libraries WPS needs.

Enable GRIB2 libraries

Find:

Edit inside the file:

```
export CONFIG_FLAGS=''
```

Replace it with:

Edit inside the file:

```
export CONFIG_FLAGS='--build-grib2-libs'
```

WPS will build its bundled zlib, libpng, and JasPer dependencies for reading GRIB2 meteorological files.

Add the Intel oneAPI MPI wrappers

Replace the small `if [ "${which_MPI}" == "intel" ]; then` block near the end of configuration with:

Edit inside the file:

```
  if [ "${which_MPI}" == "intel" ]; then
    perl -i -pe 's/^DM_FC(\s*)=(\s*)mpif90/DM_FC$1=$2mpiifort/' ${WPS_SRC_DIR}/configure.wps
    perl -i -pe 's/^DM_CC(\s*)=(\s*)mpicc/DM_CC$1=$2mpiicc/' ${WPS_SRC_DIR}/configure.wps
  elif [ "${which_MPI}" == "oneapi" ]; then
    perl -i -pe 's/^DM_FC\s*=.*/DM_FC = mpiifx/' ${WPS_SRC_DIR}/configure.wps
    perl -i -pe 's/^DM_CC\s*=.*/DM_CC = mpiicx/' ${WPS_SRC_DIR}/configure.wps
  fi
```

Preserve the surrounding script. Its existing NetCDF library handling and `int2nc` linking adjustment should remain in place.

Save and check:

Run in the terminal:

```
bash -n build_wps.sh
```

---

11. Build WPS

The WPS script now knows where to find the sources, WRF libraries, and compiler. We can build the programs that will prepare our atmospheric data.

From `Erin/prep`, run:

Run in the terminal:

```
./build_wps.sh
```

Do not add `-move` **for this exercise.** Keeping the executables in the standard WPS layout matches the paths used by the Erin preparation script.

Select **10**:

Expected display or file layout:

```
10. Linux x86_64, Intel oneAPI compilers (dmpar)
```

This is the menu choice for WPS 4.7.0 with the bundled GRIB2 libraries enabled. Check the label if your menu differs.

WPS has **no nesting compilation prompt**. After the compiler selection, it builds the libraries and preprocessing programs. It generally takes less time than WRF, but allow time for the GRIB2 dependency builds.

---

12. Check the WPS build

When WPS compilation finishes, confirm that all three preprocessing programs exist. These are the programs we will use in the next exercise.

Check the three main executables:

Run in the terminal:

```
ls -lh "$HOME/nmoa_2026/WPS/"{geogrid,ungrib,metgrid}.exe
```

The top-level files may be symbolic links. Confirm that their targets exist and are executable:

Run in the terminal:

```
for program in geogrid ungrib metgrid; do
    if test -x "$HOME/nmoa_2026/WPS/$program.exe"; then
        echo "OK: $program.exe"
    else
        echo "MISSING OR BROKEN: $program.exe"
    fi
done
```

The Erin preparation script also expects these locations:

Expected display or file layout:

```
WPS/geogrid/geogrid.exe
WPS/ungrib/ungrib.exe
WPS/metgrid/metgrid.exe
```

If any executable is missing or its link is broken, review the compilation output before proceeding. A file listing alone does not prove that every runtime dependency is available; the next exercise will test these programs with the Erin input data.

---

13. What is ready, and what comes next?

At this point, you should have:

- The pinned ROMS, WRF, and WPS sources.
- The MYNN `COARE_OPT=3.5` source setting compiled into WRF.
- Four WRF executables under `Erin/run/Build_wrf/Bin`.
- Three main WPS executables accessible through the standard WPS paths.
- Experiment-specific build scripts in `Erin/run` and `Erin/prep`.

The next exercise uses the class `get_wrf_erin.sh` in `Erin/prep`. It prepares the August 14–19, 2025 Erin case from six-hourly FNL analyses on a 20 km grid.

Before running it, students will configure its WPS, geographic-data, and `real.exe` paths. Its `real.exe` path must point to:

Expected display or file layout:

```
/home/YOUR_NETID/nmoa_2026/Erin/run/Build_wrf/Bin/real.exe
```

That exercise will generate the geographic fields, `met_em` files, and finally `wrfinput_d01` and `wrfbdy_d01`. The atmospheric forecast and the ROMS–WRF coupling setup follow afterward.